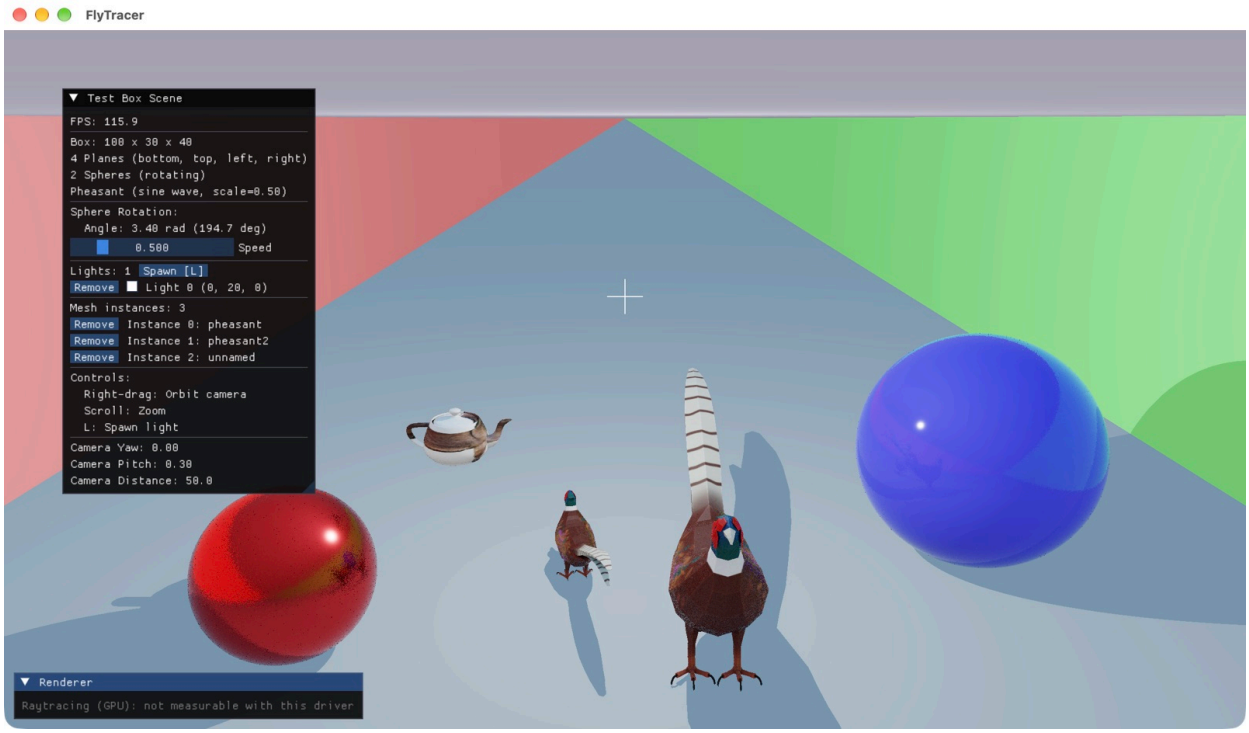
DAE • GEOMETRIC ALGEBRA

FlyTracer

Inside the Framework

How a real-time PGA raytracer is put together, from the game loop to the last Vulkan barrier



The `testbox` scene: planes, spheres and textured meshes, raytraced every frame in one compute shader, with the Dear ImGui panels drawn on top.

What you will learn

- How the engine, the game and the shader divide the work
- How a scene becomes bytes in GPU buffers
- Every Vulkan object FlyTracer creates, and why
- How one frame is recorded, synchronized and presented

What you should know already

- C++ (classes, RAII, lambdas, `std::vector`)
- The PGA basics: points, planes, lines, motors
- What a raytracer does, roughly

No Vulkan experience is needed: Part II introduces every concept as it appears in the code.

Version of October 2026. Code excerpts are taken from the repository and sometimes shortened; "`// ...`" marks a cut.

Contents

Part I The Framework **3**

1 What FlyTracer is **3**

1.1 How to read this document . . . 3

1.2 Where things live . . . 4

2 Architecture **4**

2.1 Who owns what . . . 5

3 One frame, from the top **5**

4 Writing a scene: the GameScene API **6**

4.1 Non-virtual interface . . . 6

4.2 Meshes, instances and primitives . . . 6

4.3 An example . . . 6

5 From scene to bytes **7**

5.1 GPU structs . . . 8

5.2 The shared interface . . . 8

5.3 Bounding volume hierarchies . . . 9

Part II The Vulkan Code **10**

6 Vulkan in one chapter **10**

7 VulkanHelpers: the building blocks **10**

7.1 Errors . . . 11

7.2 Enumerations . . . 11

7.3 Owning handles . . . 11

7.4 Create-info structs . . . 12

7.5 Buffers and images . . . 12

7.6 Submitting work . . . 12

7.7 Barriers . . . 13

8 VulkanContext: instance and device **13**

8.1 The instance . . . 13
8.2 Choosing a GPU . . . 13
8.3 The logical device and its features . . . 14

9 Swapchain **14**

10 RaytracingPass: the compute pipeline **15**
10.1 Resources . . . 15
10.2 Descriptors and pipeline . . . 15
10.3 Uploading meshes . . . 16
10.4 Per-frame buffers . . . 16
10.5 Recording the dispatch . . . 16

11 VulkanRenderer: frames in flight and synchronization **17**
11.1 Frames in flight . . . 17
11.2 BeginFrame . . . 17
11.3 EndFrame: one command buffer . . . 18
11.4 Submit and present . . . 18

12 ImGui with dynamic rendering **19**

13 Measuring: GpuTimer and Benchmark **20**

Part III The GPU Side **21**

14 The compute shader **21**
14.1 Files . . . 21
14.2 One invocation per pixel . . . 21
14.3 The camera is a motor . . . 22
14.4 Rays into instances . . . 22

15 Testing and debugging **22**

16 Exercises **23**

A Vulkan calls by class **23**

B Further reading **24**

PART I

The Framework

1 What FlyTracer is

FlyTracer is a real-time raytracer written to teach Projective Geometric Algebra (PGA). Everything that moves (the camera, mesh instances, reflections) is a PGA element: points are `TriVectors`, planes are `Vectors`, lines are `BiVectors` and rigid motions are `Motors`, all from the header-only FlyFish library. There are no matrices, not on the CPU and not on the GPU.

The image is computed by a single Vulkan compute shader, engine/shaders/raytracer.comp. Every frame it shoots one ray per pixel, follows reflections, adds the result to a running average and writes a finished image that is copied to the window. The C++ side prepares the scene, uploads it, and orchestrates the GPU work.

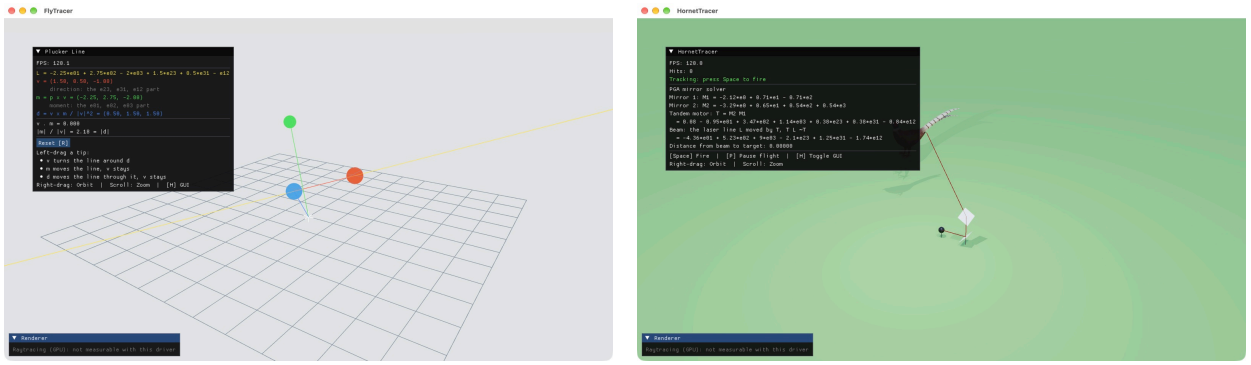


Figure 1: Two of the teaching scenes. Left: `plucker`, a line in Plücker coordinates with its direction, moment and support vector as debug lines. Right: `hornet`, two mirrors solved with PGA steer a laser beam onto a target.

1.1 How to read this document

Part I, the framework follows one frame through the C++ code: the main loop, the scene API that students program against, and how the scene is encoded into GPU structs.

Part II, the Vulkan code explains every Vulkan object FlyTracer creates, in the order they are created, and then the synchronization of a frame in detail.

Part III, the GPU walks through the compute shader and ends with testing, debugging and exercises.

Coloured boxes mark **key ideas**, **pitfalls** and **things to try**.

1.2 Where things live

Path	Contents
engine/include, engine/src	The engine: two static libraries, `FlyTracerCore` and `FlyTracerEngine`
engine/shaders	The GLSL compute shader and its include files, compiled to SPIR-V by `glslc`
game/	The executable: `main.cpp`, the scene registry and the teaching scenes
game/resources	OBJ meshes and textures, copied next to the executable
tests/	Google Test unit tests, GPU tests and golden reference images
docs/	The student documentation (MkDocs) and this overview

```
cmake -B build -DCMAKE_BUILD_TYPE=Release   # Debug enables the validation layer
cmake --build build                          # executable, shaders, resources and tests
./build/bin/FlyTracer --scene testbox        # also debug, bireflection, birot, plucker, ...
ctest --test-dir build
```

2 Architecture

FlyTracer is split into layers that only depend downwards (Figure 2). The split is not cosmetic: each boundary is something you can test or replace on its own.

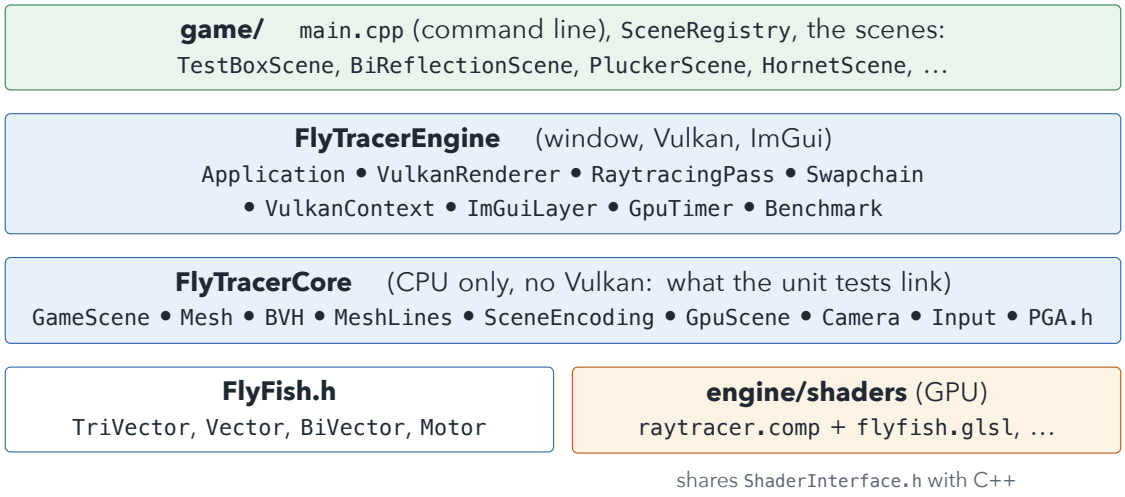


Figure 2: The layers. Each one only uses the layers below it.

FlyTracerCore knows the scene: meshes, bounding volume hierarchies, the camera, and how all of it is laid out in bytes for the GPU. It never includes `vulkan.h`, so `FlyTracer_Test` runs on any machine, also without a GPU.

FlyTracerEngine owns the window and everything Vulkan. It only reads the scene through the const getters of `GameScene`.

The game contains no engine code at all: a scene derives from `GameScene` and uses its protected API.

The shader is a separate program. The contract between it and C++ is ShaderInterface.h (binding numbers and limits, `#defines` both languages include) and the structs in GpuScene.h, which a GPU test checks against the GLSL structs.

2.1 Who owns what

Ownership follows the RAII idiom: an object's members are constructed in declaration order and destroyed in reverse. FlyTracer uses this deliberately. `VulkanRenderer` declares the context first, so it is destroyed last, after everything that was created from it:

```
class VulkanRenderer {
    // ...
    SDL_Window* m_window;
    // Declaration order is construction order; destruction runs backwards, so the context goes last
    VulkanContext m_context;
    Swapchain m_swapchain;
    RaytracingPass m_raytracing;
    ImGuiLayer m_imgui;
    std::array<Frame, RaytracingPass::kFramesInFlight> m_frames;
    // ...
};
```

`Application` owns the SDL window, the `VulkanRenderer`, the `GameScene` and the `Input` state. There are no global objects and no singletons.

3 One frame, from the top

The whole program is a loop in `Application::run` (engine/src/Application.cpp):

```
while (m_running) {
    const double deltaTime = std::min(/* seconds since the last frame */, kMaxDeltaTime);
    handleEvents();                          // SDL events -> Input (keys, mouse, resize, quit)
    // ...
    m_gameScene->HandleInput(m_input);       // the scene reacts to input
    m_input.EndFrame();
    m_gameScene->Update(static_cast<float>(deltaTime));   // the scene animates
    render(static_cast<float>(deltaTime));
}
m_renderer->WaitIdle();

void Application::render(float deltaTime) {
    if (!m_renderer->BeginFrame()) return;   // no image available (minimized, resizing)
    m_gameScene->DrawGui();                  // ImGui windows of the scene
    drawRendererStats(deltaTime);
    m_renderer->EndFrame(*m_gameScene);      // record, submit and present
}
```

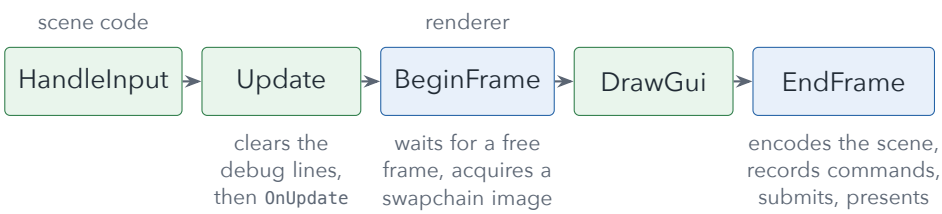


Figure 3: The steps of one frame. Green steps call into the scene, blue ones are the renderer.

Two details are worth noticing. First, `deltaTime` is clamped, so a frame that took very long (a breakpoint, dragging the window) does not make animations jump. Second, ImGui's frame is opened in `BeginFrame` and closed in `EndFrame`; `DrawGui` sits in between so scenes can create windows

with plain `ImGui::Begin`.

4 Writing a scene: the GameScene API

4.1 Non-virtual interface

`GameScene` (engine/include/GameScene.h) uses the *non-virtual interface* idiom: the engine calls public, non-virtual functions, which call protected virtual hooks that scenes override.

Engine calls	Scene overrides	What the wrapper adds
`Init()`	`OnInit()`	
`HandleInput(input)`	`OnInput(input)`	
`Update(dt)`	`OnUpdate(dt)`	FPS counter; clears last frame's debug lines
`DrawGui()`	`OnGui()`	
`Shutdown()`	`OnShutdown()`	

The wrapper guarantees an order a scene cannot forget: debug lines are always cleared before `OnUpdate`, so a scene simply draws the lines it wants this frame.

4.2 Meshes, instances and primitives

A *mesh* is geometry loaded once (`LoadMesh("pheasant.obj", "pheasant.png")` or `AddMesh` for one built in code); its bottom-level BVH is built when it is added. An *instance* places a mesh in the world. Many instances can share one mesh, which is why the two pheasants in `testbox` cost almost nothing extra.

```
struct MeshInstance {
    uint32_t meshId{0};
    Motor transform{1.0f, 0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 0.0f};
    float scale{1.0f};
    std::optional<Vector> mirror;   // a reflection after the motor: an odd versor
    bool visible{true};
    std::string name;
};
```

A local point p goes to $M\,(s\,p)\,\widetilde{M}$, followed by the reflection in the mirror plane when there is one. Spheres, planes and lights are analytic primitives, added with `AddSphere`, `AddPlane` and `AddPointLight` and friends. All `Add*` functions return an id that stays valid when other objects are removed.

4.3 An example

This is most of game/src/TestBoxScene.cpp. Note that animation is pure PGA: a motor is built from an angle and a line, and applied with a sandwich.

```
void TestBoxScene::OnInit() {
    AddPlane(Vector(0.0f, 0.0f, 1.0f, 0.0f), Scene::Material::Lambert(Scene::Color(0.6f, 0.6f,
     0.6f)));
    // ... three more planes
    m_sphere1Id = AddSphere(TriVector(m_sphere1Radius, m_sphere1Height, 0.0f), 5.0f,
                            Scene::Material::Metal(Scene::Color(0.9f, 0.3f, 0.3f), 0.2f));
```

```
    m_pheasantMeshId = LoadMesh("pheasant.obj", "pheasant.png");
    AddMeshInstance(m_pheasantMeshId, TriVector(0.0f, m_pheasantHeight, 0.0f), "pheasant");
    AddPointLight(TriVector(0.0f, 20.0f, 0.0f), Scene::Color(1.0f, 1.0f, 1.0f), 2.0f, 100.0f);
}

void TestBoxScene::OnUpdate(float deltaTime) {
    m_rotationAngle = std::fmod(m_rotationAngle + m_rotationSpeed * deltaTime, 2.0f * kPi);
    const Motor rotation = Motor::Rotation(m_rotationAngle * RAD_TO_DEG,
                                           BiVector(0.0f, 0.0f, 0.0f, 0.0f, 1.0f, 0.0f));
    if (auto* sphere = GetSphere(m_sphere1Id)) {
        sphere->center = ApplyMotor(rotation, TriVector(m_sphere1Radius, m_sphere1Height, 0.0f));
    }
    if (auto* pheasant = FindInstance("pheasant")) {
        const float sway = m_pheasantSway * std::sin(m_pheasantTime);
        pheasant->transform = rotation * TranslationMotor(sway, m_pheasantHeight, 0.0f);
    }
}

void TestBoxScene::OnInput(const Input& input) {
    m_orbit.HandleInput(input);                  // right-drag orbit, scroll zoom
    SetCamera(m_orbit.Eye(), m_orbit.target);
}
```

Pitfall: motor order

`A * B` applies `B` first. `rotation * TranslationMotor(...)` translates the pheasant and then rotates the result about the y-axis, so it swings around the origin. Swap the factors and it turns on the spot at its new position instead.

Try it: add a scene

Create game/include/MyScene.h and game/src/MyScene.cpp deriving from `GameScene`, add the `.cpp` to the executable in CMakeLists.txt, and register the name in game/src/SceneRegistry.cpp. Run it with `--scene <name>`. The scene test will record a reference image for it the first time it runs.

5 From scene to bytes

The GPU cannot read a `std::vector<std::unique_ptr<Mesh>>`. Before each frame the scene is *encoded*: flattened into arrays of plain structs whose memory layout matches the GLSL structs byte for byte. This happens entirely in `FlyTracerCore` (SceneEncoding.h, GpuScene.h), so it is unit-tested without a GPU.

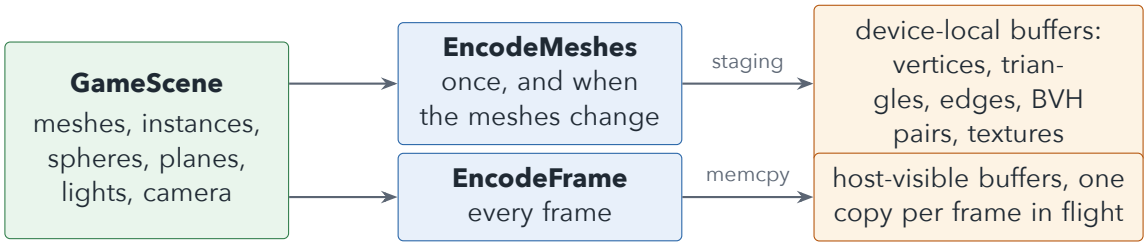


Figure 4: Two kinds of encoding. Geometry rarely changes, so it lives in fast device-local memory. Everything else is small and is rewritten every frame.

5.1 GPU structs

Each scene type has a `Gpu::` twin and an `Encode` function that converts it: colours go from sRGB to linear, planes and motors are normalized, a motor is split into a rotor part and a dual part of four floats each.

```
namespace Gpu {
struct Sphere {
    Float3 center;          // std::array<float, 3>
    float radius;
    uint32_t materialIndex;
    uint32_t _pad[3];       // rounds the struct up to 32 bytes, like std430 does in GLSL
};
static_assert(sizeof(Sphere) == 32 && offsetof(Sphere, materialIndex) == 16);
}
```

The `static_asserts` pin the layout on the C++ side. On the GLSL side the matching struct lives in common.glsl. `FlyTracer_ShaderTest` checks that both sides agree. It fills every 32-bit word of the C++ data with its binding number and position, and a test shader (tests/shaders/interface_test.comp) reads that data through the raytracer's own GLSL declarations and writes back what it sees. A member at another offset, an array with another stride or a buffer at another binding shows up as a wrong word, so adding a field on one side only fails the test.

> **Key idea: std430 versus scalar layout**
>
> GLSL's default `std430` layout aligns a `vec3` to 16 bytes. That is why many structs have explicit padding. The geometry buffers that are read most (edges, edge-triangle records, BVH node pairs) instead use the `scalar` layout (`GL_EXT_scalar_block_layout`), which packs like C++ without padding and makes those buffers smaller and faster. It is a device feature (`scalarBlockLayout`) that `VulkanContext` enables.

5.2 The shared interface

engine/include/ShaderInterface.h contains only `#defines`, so C++ and GLSL can both include it (`glslc` gets `-I engine/include`):

Binding	Name	Contents	Updated
0	`ACCUMULATION_IMAGE`	`rgba32f` running average of linear radiance	by the shader
1	`OUTPUT_IMAGE`	`rgba8` tone-mapped, sRGB-encoded result	by the shader
2–7	`VERTICES` … `MESH_ROOT_PAIRS`	all meshes concatenated, their BVHs	when meshes change
8–14	`MATERIALS` … `DEBUG_LINES`	materials, primitives, lights, instances, top-level BVH	every frame
15	`TEXTURES`	64 combined image samplers, slot 0 is white	when meshes change

Other limits defined there: `MAX_TEXTURES` (64), `BVH_STACK_SIZE` (32), `WORKGROUP_SIZE` (8, so a workgroup is 8×8 pixels) and the `REVERSED_EDGE` bit.

5.3 Bounding volume hierarchies

Testing every ray against every triangle would be far too slow. FlyTracer uses a *two-level* hierarchy:

- a **bottom-level BVH** per mesh, in object space, built once with a binned surface area heuristic (`BuildBVH` in BVH.cpp), its depth limited so the shader's stack of `BVH_STACK_SIZE` entries never overflows;
- a **top-level BVH** over the instances in world space, rebuilt every frame in `EncodeFrame`, so instances can move freely without touching the meshes.

On the GPU a node is stored as a `NodePair`: the bounding boxes of *both* children of an internal node in one 64-byte record. One memory read gives the shader both boxes it must test, and it can visit the nearer child first. Triangles are stored in the BVH's leaf order, so a leaf is just a range `[index, index + count)`.

PART II

The Vulkan Code

6 Vulkan in one chapter

Vulkan is an explicit API: it does almost nothing implicitly. You create every object yourself, record GPU work into command buffers, submit them to a queue, and tell the GPU which work must wait for which. In return you get predictable performance and you see everything the driver does. Table 1 lists the concepts FlyTracer uses and where each one lives.

Concept	What it is	In FlyTracer
Instance	The connection between the application and the Vulkan loader; validation layers hook in here.	`VulkanContext`
Surface	What a window looks like to Vulkan. Created by SDL.	`VulkanContext`
Physical device	One GPU, with its properties, features and queue families.	`VulkanContext`
Device	The logical device: your session with one GPU, with the features you enabled.	`VulkanContext`
Queue	Where command buffers are submitted. FlyTracer uses one.	`VulkanContext`
Command pool, buffer	Memory for recorded commands; a buffer is a list of commands to submit.	`VulkanContext`, `VulkanRenderer`
Buffer, image, memory	Raw GPU data. Memory is allocated separately and bound to them.	`VulkanHelpers`
Image view	How a shader or attachment sees an image (format, mip levels).	`VulkanHelpers::Image`
Image layout	The memory arrangement an image is in; operations require specific layouts.	everywhere
Descriptor set (layout, pool)	The table of buffers and images a shader can access, by binding number.	`RaytracingPass`
Push constants	A few bytes (here 72) passed directly with a command, no buffer needed.	`RaytracingPass`
Pipeline, pipeline layout	The compiled shader plus the shape of its inputs.	`RaytracingPass`
Barrier	"Work after this point waits for that work before it", and layout changes.	`VulkanHelpers`
Semaphore	GPU-to-GPU signal between submissions and the presentation engine.	`VulkanRenderer`, `Swapchain`
Fence	GPU-to-CPU signal: the CPU can wait until a submission has finished.	`VulkanRenderer`
Swapchain	The images the window shows, owned by the presentation engine.	`Swapchain`
Query pool	GPU timestamps, for measuring.	`GpuTimer`

Table 1: The Vulkan concepts used by FlyTracer.

7 VulkanHelpers: the building blocks

engine/include/VulkanHelpers.h keeps the rest of the Vulkan code short. Knowing its pieces makes the other classes easy to read.

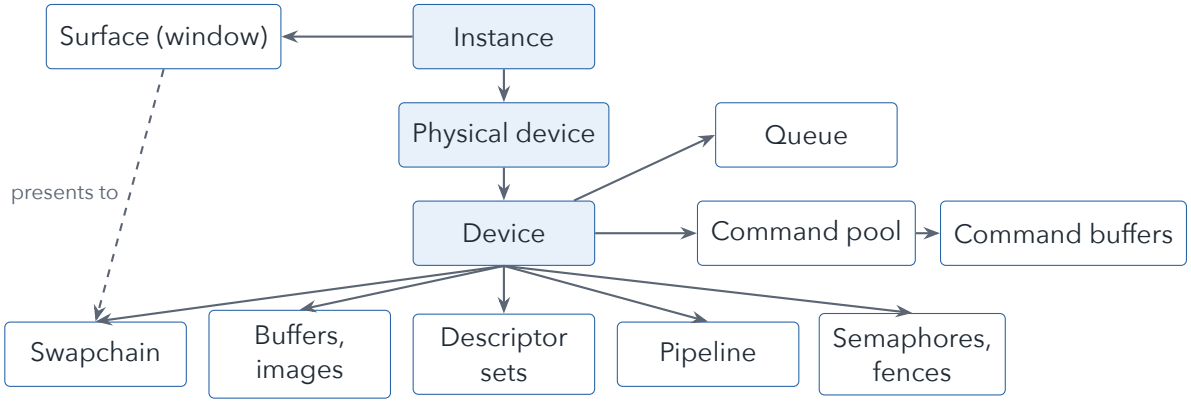


Figure 5: What is created from what. Everything below the device is destroyed before the device.

7.1 Errors

Almost every Vulkan function returns a `VkResult`. `check` turns failure into an exception that names the call:

```
inline void check(VkResult result, const char* what) {
    if (result != VK_SUCCESS) {
        throw std::runtime_error(std::string(what) + " failed (VkResult " + std::to_string(result) +
    ")");
    }
}
check(vkCreateDevice(m_physicalDevice, &createInfo, nullptr, &m_device), "vkCreateDevice");
```

7.2 Enumerations

Vulkan returns lists in two calls: one to get the count, one to fill an array. `enumerate` does both, for any such function, thanks to a variadic template:

```
template<typename T, typename Function, typename... Args>
[[nodiscard]] std::vector<T> enumerate(Function function, Args... args) {
    uint32_t count = 0;
    function(args..., &count, nullptr);
    std::vector<T> elements(count);
    function(args..., &count, elements.data());
    elements.resize(count);
    return elements;
}
const auto devices = enumerate<VkPhysicalDevice>(vkEnumeratePhysicalDevices, m_instance);
```

7.3 Owning handles

Vulkan handles are plain values; nothing destroys them for you. `DeviceHandle` is a move-only owner, like `std::unique_ptr`, that calls the right `vkDestroy*` function. The destroy function is a template parameter, so each alias is a distinct type:

```
template<typename Handle, auto Destroy>
class DeviceHandle {
public:
    DeviceHandle(VkDevice device, Handle handle) noexcept : m_device(device), m_handle(handle) {}
```

```
    DeviceHandle(DeviceHandle&& other) noexcept;              // takes over the handle
    ~DeviceHandle() { reset(); }                              // Destroy(m_device, m_handle, nullptr)
    [[nodiscard]] Handle get() const noexcept { return m_handle; }
    // ...
};
using UniqueBuffer = DeviceHandle<VkBuffer, vkDestroyBuffer>;
using UniqueFence = DeviceHandle<VkFence, vkDestroyFence>;
using UniquePipeline = DeviceHandle<VkPipeline, vkDestroyPipeline>;
// ... 13 aliases in all
```

7.4 Create-info structs

Every Vulkan object is created from a `Vk*CreateInfo` struct whose first member `sType` names the struct. FlyTracer fills them with C++20 *designated initializers*: members you do not name are zero, which is the right default for almost every Vulkan field.

```
const VkFenceCreateInfo createInfo{
    .sType = VK_STRUCTURE_TYPE_FENCE_CREATE_INFO,
    .flags = signaled ? VK_FENCE_CREATE_SIGNALED_BIT : VkFenceCreateFlags{0},
};
```

> **Pitfall**
>
> Designated initializers must follow the declaration order of the struct, so `.pNext` comes before `.flags`, which comes before the rest. The compiler tells you when the order is wrong.

7.5 Buffers and images

`Buffer` and `Image` bundle a handle, its own memory and (for images) a view. The memory type is chosen with `findMemoryType`, which looks for a type allowed by the resource that has the requested properties.

- **Device-local** memory is the fastest for the GPU but the CPU cannot write it directly. Static mesh data and all images live here.
- **Host-visible, host-coherent** memory can be mapped: the CPU writes it with `memcpy` and the GPU sees the bytes without an explicit flush. A `Buffer` in such memory stays mapped for its whole life.
- `hostWriteMemoryProperties` asks for memory that is *both* when the GPU has it, as on Apple silicon's unified memory or desktop GPUs with resizable BAR. The per-frame scene buffers use it.

7.6 Submitting work

`submit` wraps `vkQueueSubmit2` for one command buffer with an optional wait semaphore, signal semaphore and fence. `CommandSubmitter` is for one-time work that the CPU waits for: it records, submits, waits for the queue to go idle, and returns the time that took.

```
VulkanHelpers::CommandSubmitter(device, commandPool, queue).Submit([&](VkCommandBuffer cmd) {
    // record commands; Submit returns after the GPU executed them
});
```

`StagingUploader` uploads data to device-local memory. Since the CPU cannot write there, each upload is written to a temporary host-visible *staging* buffer, and a copy command moves it to the final buffer. All copies go into one command buffer that is submitted once, in `submit()`, after which the staging buffers are freed. `uploadTexture` also generates the mip chain by blitting each level into the next, half the size.

7.7 Barriers

`transitionImageLayout2` records one `VkImageMemoryBarrier2`: it moves an image from one layout to another, and it makes the commands after it at `dstStage` wait for the commands before it at `srcStage`, with the memory written by `srcAccess` made visible to `dstAccess`. Section 11 uses it for every step of a frame.

8 VulkanContext: instance and device

engine/src/VulkanContext.cpp creates the objects everything else depends on. Its constructor takes an optional SDL window; without one (benchmark, GPU tests) no surface and no swapchain are created.

8.1 The instance

```
const VkApplicationInfo appInfo{
    .sType = VK_STRUCTURE_TYPE_APPLICATION_INFO,
    .pApplicationName = "FlyTracer",
    .pEngineName = "FlyTracer",
    .apiVersion = VK_API_VERSION_1_4,
};
```

The instance extensions are the ones SDL needs to make a surface for the window (`SDL_Vulkan_GetInstanceExtensions`). In Debug builds the `VK_LAYER_KHRONOS_validation` layer is enabled when installed, with a debug messenger that prints warnings and errors in colour. The messenger's create info is also chained into the instance create info (through `pNext`), so messages from `vkCreateInstance` itself are reported too. Setting `FLYTRACER_GPU_VALIDATION=1` additionally turns on GPU-assisted validation, which checks shader accesses at run time.

8.2 Choosing a GPU

`selectPhysicalDevice` loops over all GPUs and keeps the best one that

1. supports Vulkan 1.3 (synchronization2 and dynamic rendering are core there), and
2. has a queue family that can do graphics *and* compute, and present to the window.

Discrete GPUs score highest, then integrated, virtual and CPU implementations.

> **Key idea: one queue**
>
> FlyTracer does all its work (the compute dispatch, the copy to the swapchain, the ImGui draw and presenting) on a single queue. Commands on one queue start in submission order, so with barriers between them there is no need for queue ownership transfers or cross-queue semaphores. A second, async compute queue could overlap work, but would make the syn-

chronization much harder to teach.

8.3 The logical device and its features

Vulkan features are off until you enable them. FlyTracer enables exactly what it uses, by chaining feature structs through `pNext`:

```
VkPhysicalDeviceVulkan13Features vulkan13Features{
    .sType = VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_VULKAN_1_3_FEATURES,
    .synchronization2 = VK_TRUE,     // vkCmdPipelineBarrier2, vkQueueSubmit2
    .dynamicRendering = VK_TRUE,     // ImGui draws without a render pass
};
VkPhysicalDeviceVulkan12Features vulkan12Features{
    .sType = VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_VULKAN_1_2_FEATURES,
    .pNext = &vulkan13Features,
    .shaderSampledImageArrayNonUniformIndexing = VK_TRUE,  // textures[index] varies per pixel
    .scalarBlockLayout = VK_TRUE,                          // tightly packed buffers
};
const VkDeviceCreateInfo createInfo{
    .sType = VK_STRUCTURE_TYPE_DEVICE_CREATE_INFO,
    .pNext = &vulkan12Features,
    // one queue, and VK_KHR_swapchain when there is a window
};
```

Finally the context gets the queue and creates a command pool with `VK_COMMAND_POOL_CREATE_RESET_COMMAND_BUFFER_BIT`, so each frame's command buffer can be reset and recorded again.

> **Pitfall: enabling everything**
>
> It is tempting to enable every feature the GPU supports. Some, such as `robustBufferAccess`, add bounds checks to every memory access and cost performance. Enable what you use.

9 Swapchain

A window cannot be drawn into directly. The *presentation engine* owns a small set of images; each frame you acquire one, render into it, and hand it back to be shown. engine/src/Swapchain.cpp makes these choices:

Format `B8G8R8A8_UNORM` (or `R8G8B8A8_UNORM`) with the sRGB colour space. Deliberately *not* an `_SRGB` format: the shader already encodes sRGB itself, and ImGui's colours are sRGB values. An `_SRGB` swapchain would encode a second time and wash the image out.

Present mode With vsync `FIFO`, which waits for the display and is always available; without, `MAILBOX` or else `IMMEDIATE`.

Image count One more than the minimum, so the application rarely waits for the presentation engine to release an image.

Usage `TRANSFER_DST` (the raytraced image is blitted in) and `COLOR_ATTACHMENT` (ImGui draws on top).

Extent The surface's current size, or the window size in pixels on platforms that leave it to the application.

For every image the swapchain also creates an image view (for ImGui) and a *render-finished*

semaphore. That semaphore belongs to the image rather than to a frame in flight, because a present can still be waiting on it after the frame's fence has signalled; reusing it for the next frame could signal a semaphore that is still in use.

When the window is resized, `Recreate` builds a new swapchain, passing the old one as `oldSwapchain` so the driver can hand over resources. The caller (`VulkanRenderer`) first waits for the GPU to be idle.

10 RaytracingPass: the compute pipeline

engine/src/RaytracingPass.cpp owns everything the compute shader needs. It is also used without a window by the benchmark and the GPU tests.

10.1 Resources

Resource	Memory	Purpose
`m_accumulation`	device-local, `rgba32f`	Running average of linear radiance, kept between frames
`m_output`	device-local, `rgba8`	Tone-mapped sRGB result, the source of the blit
mesh buffers (6)	device-local, uploaded by staging	Vertices, triangles, edges, edge-triangle records, BVH pairs, root pairs
`m_textures`	device-local, `R8G8B8A8_SRGB`, mipmapped	Up to 64; slot 0 is a white 1×1 image
`FrameBuffers` ($\times 2$)	host-visible	Materials, spheres, planes, lights, instances, top-level BVH, debug lines
descriptor sets ($\times 2$)		One per frame in flight, pointing at that frame's buffers

10.2 Descriptors and pipeline

Every binding is a storage buffer except the two storage images and the texture array. The layout is built from the numbers in ShaderInterface.h:

```
std::array<VkDescriptorSetLayoutBinding, BINDING_COUNT> bindings{};
for (uint32_t binding = 0; binding < BINDING_COUNT; ++binding) {
    bindings[binding] = {binding, VK_DESCRIPTOR_TYPE_STORAGE_BUFFER, 1, VK_SHADER_STAGE_COMPUTE_BIT,
     nullptr};
}
bindings[BINDING_ACCUMULATION_IMAGE].descriptorType = VK_DESCRIPTOR_TYPE_STORAGE_IMAGE;
bindings[BINDING_OUTPUT_IMAGE].descriptorType = VK_DESCRIPTOR_TYPE_STORAGE_IMAGE;
bindings[BINDING_TEXTURES].descriptorType = VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER;
bindings[BINDING_TEXTURES].descriptorCount = kMaxTextures;
m_descriptorSetLayout = VulkanHelpers::createDescriptorSetLayout(device, bindings);
```

A descriptor pool is sized for two sets of that layout, and one set is allocated per frame in flight. The pipeline layout combines the set layout with a push constant range the size of `Gpu::PushConstants` (72 bytes, below the 128 bytes Vulkan guarantees). The pipeline itself is created from `raytracer.comp.spv`, which CMake compiled with `glslc`. A linear, repeating sampler with mipmapping is shared by all textures.

10.3 Uploading meshes

`UploadMeshes` runs once after the scene's `OnInit`, and again from `Record` when `MeshesChanged` reports added meshes or changed counts or textures. It waits for the GPU to be idle (the old buffers may still be in use), encodes all meshes, uploads the six buffers and the textures with one `StagingUploader`, and writes their descriptors into both sets. Textures get $\lfloor \log_2(\max(w, h)) \rfloor + 1$ mip levels (`std::bit_width`), which the shader uses to pick a sharp but not aliased level per pixel. Unused texture slots point at the white texture, so every descriptor in the array is valid.

10.4 Per-frame buffers

The small per-frame buffers grow on demand: when the data no longer fits, a buffer of twice the size is created and its descriptor is rewritten.

```
template<typename T>
void RaytracingPass::writeFrameBuffer(VulkanHelpers::Buffer& buffer, VkDescriptorSet set,
                                      uint32_t binding, const std::vector<T>& data) {
    const VkDeviceSize dataSize = sizeof(T) * data.size();
    if (buffer.buffer() == VK_NULL_HANDLE || buffer.size() < dataSize) {
        const VkDeviceSize capacity = std::max({dataSize, 2 * buffer.size(),
    VkDeviceSize{sizeof(T)}});
        buffer = VulkanHelpers::Buffer(/* ... */ capacity, VK_BUFFER_USAGE_STORAGE_BUFFER_BIT,

    VulkanHelpers::hostWriteMemoryProperties(m_context.PhysicalDevice()));
        VulkanHelpers::writeStorageBufferDescriptor(m_context.Device(), set, binding,
    buffer.buffer());
    }
    if (dataSize > 0) buffer.write(data.data(), dataSize);
}
```

> **Key idea: why two copies**
>
> While the GPU executes frame n, the CPU already writes the data of frame $n+1$. If both used the same buffer, the CPU could overwrite data the GPU is still reading. With one copy per frame in flight, the CPU only writes a frame's buffers (and replaces its descriptor set's entries) after waiting on that frame's fence, so the GPU is done with them.

10.5 Recording the dispatch

`Record` is called once per frame with the frame's command buffer:

1. Re-upload the meshes if they changed, then `EncodeFrame` and write the seven per-frame buffers.
2. Build the push constants: the camera motor (from `CameraMotor`, a motor built from joins of the eye, target and up), the field of view, counts, bounces, exposure.
3. Decide whether accumulation continues. The bytes of everything that affects the image (materials, primitives, lights, instances and the push constants) are compared with last frame's. If anything changed, `sampleIndex` goes back to 0 and the average restarts; otherwise it counts up. Debug lines are drawn after accumulation, so they are left out of the comparison.
4. Record a barrier on the accumulation image, bind the pipeline and descriptor set, push the constants, and dispatch one invocation per pixel:

```
vkCmdBindPipeline(cmdBuffer, VK_PIPELINE_BIND_POINT_COMPUTE, m_pipeline.get());
vkCmdBindDescriptorSets(cmdBuffer, VK_PIPELINE_BIND_POINT_COMPUTE, m_pipelineLayout.get(),
                        0, 1, &frame.descriptorSet, 0, nullptr);
vkCmdPushConstants(cmdBuffer, m_pipelineLayout.get(), VK_SHADER_STAGE_COMPUTE_BIT, 0,
                   sizeof(pushConstants), &pushConstants);
m_timer.Begin(cmdBuffer, frameIndex);
vkCmdDispatch(cmdBuffer, (extent.width + WORKGROUP_SIZE - 1) / WORKGROUP_SIZE,
              (extent.height + WORKGROUP_SIZE - 1) / WORKGROUP_SIZE, 1);
m_timer.End(cmdBuffer, frameIndex);
```

The division rounds up, so the workgroups cover the image even when its size is not a multiple of 8; the shader returns early for invocations outside the image.

11 VulkanRenderer: frames in flight and synchronization

engine/src/VulkanRenderer.cpp ties everything together. It is the place where most Vulkan bugs would show up, so it is worth reading slowly.

11.1 Frames in flight

```
struct Frame {
    VkCommandBuffer commandBuffer{VK_NULL_HANDLE};
    VulkanHelpers::UniqueSemaphore imageAvailable;   // signalled when the acquired image is ready
    VulkanHelpers::UniqueFence inFlight;             // signalled when the frame's commands finished
};
std::array<Frame, RaytracingPass::kFramesInFlight> m_frames;   // kFramesInFlight == 2
```

With two frames in flight the CPU can prepare frame $n+1$ while the GPU renders frame n; neither waits for the other unless one is more than a frame ahead (Figure 6).

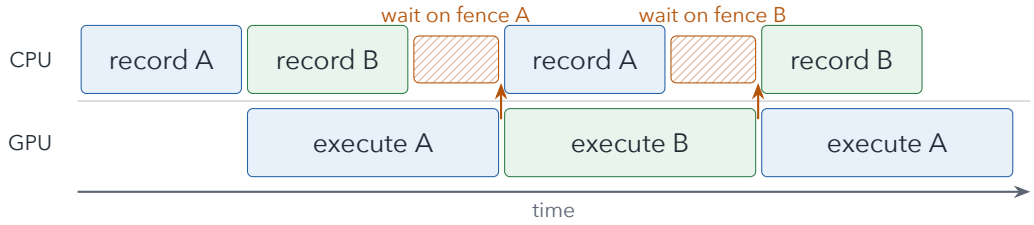


Figure 6: Two frames in flight, here with the GPU as the bottleneck. Before the CPU records frame A again it waits on A's fence, while frame B is already queued. The CPU never gets more than two frames ahead.

11.2 BeginFrame

1. Skip the frame if the window is minimized (zero pixels); recreate the swapchain if it was marked out of date.
2. **Wait on this frame's fence.** After this, the frame's command buffer, per-frame buffers, descriptor set and timestamp queries are free to reuse.
3. **Acquire a swapchain image** with `vkAcquireNextImageKHR`. It returns an index immediately, but the image may still be read by the display: the `imageAvailable` semaphore signals when it really is free. `VK_ERROR_OUT_OF_DATE_KHR` means the window changed and the frame is

skipped; `VK_SUBOPTIMAL_KHR` still renders, but recreates next frame.

4. If the acquired image was last used by the *other* frame, wait on that frame's fence too (`m_imagesInFlight`). This happens when the swapchain returns images out of order.
5. Reset the fence and start the ImGui frame.

11.3 EndFrame: one command buffer

The whole frame is recorded into one command buffer:

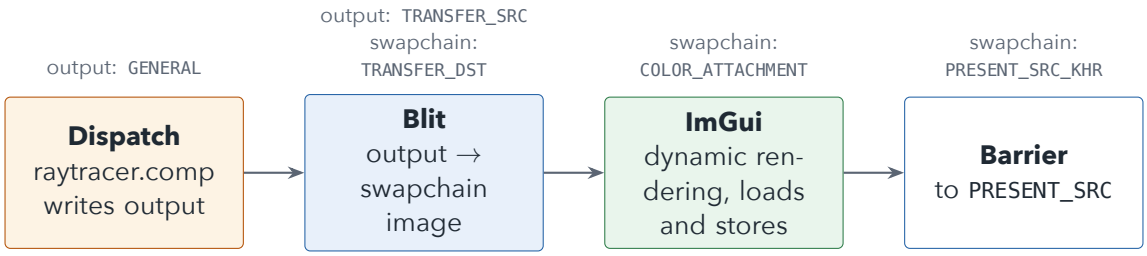


Figure 7: The commands of one frame, with the layout of the image each one works on.

The compute shader cannot write the swapchain image directly (its format is not guaranteed to support storage), and the raytracer's image is RGBA while the swapchain is usually BGRA. `vkCmdBlitImage` copies and converts in one command. Every arrow above is a barrier. The prefixes `VK_PIPELINE_STAGE_2_`, `VK_ACCESS_2_` and `VK_IMAGE_LAYOUT_` are left out in Table 2.

	Image	Layout	Waits for (src)	Before (dst)	Why
1	accumulation	GENERAL → GENERAL	COMPUTE_SHADER, STORAGE_WRITE	COMPUTE_SHADER, STORAGE_READ|WRITE	The previous frame's dispatch wrote the average this one reads.
2	output	GENERAL → TRANSFER_SRC	COMPUTE_SHADER, STORAGE_WRITE	BLIT, TRANSFER_READ	The blit reads what the shader wrote.
3	swapchain	UNDEFINED → TRANSFER_DST	BLIT, NONE	BLIT, TRANSFER_WRITE	Old contents are discarded. The src stage equals the semaphore's wait stage, so the transition happens after the acquire.
4	swapchain	TRANSFER_DST → COLOR_ATTACHMENT	BLIT, TRANSFER_WRITE	COLOR_ATTACHMENT_OUTPUT, READ|WRITE	ImGui loads the blitted pixels and blends over them.
5	output	TRANSFER_SRC → GENERAL	BLIT, TRANSFER_READ	COMPUTE_SHADER, STORAGE_WRITE	The next dispatch may only overwrite the output after the blit read it.
6	swapchain	COLOR_ATTACHMENT → PRESENT_SRC	COLOR_ATTACHMENT_OUTPUT, WRITE	NONE	The presentation engine needs `PRESENT_SRC`; the semaphore makes the writes available.

Table 2: The barriers of one frame, in recording order (1 is in `RaytracingPass::Record`, 2–5 in `recordBlitToSwapchain`, 6 in `EndFrame`).

11.4 Submit and present

```
const VkSemaphoreSubmitInfo waitInfo{
    .sType = VK_STRUCTURE_TYPE_SEMAPHORE_SUBMIT_INFO,
    .semaphore = frame.imageAvailable.get(),
```

```
    .stageMask = VK_PIPELINE_STAGE_2_BLIT_BIT,          // only the blit has to wait for the image
};
const VkSemaphoreSubmitInfo signalInfo{
    .sType = VK_STRUCTURE_TYPE_SEMAPHORE_SUBMIT_INFO,
    .semaphore = renderFinished,                          // of this swapchain image
    .stageMask = VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT,   // includes the transition to PRESENT_SRC
};
VulkanHelpers::submit(m_context.Queue(), cmdBuffer, &waitInfo, &signalInfo, frame.inFlight.get());
```

Three synchronization objects work together:

- `imageAvailable`: presentation engine → GPU. The wait stage is `BLIT`, not the top of the pipe, so the raytracing dispatch runs while the display still shows the image; only the blit waits.
- `renderFinished`: GPU → presentation engine. `vkQueuePresentKHR` waits on it.
- `inFlight` fence: GPU → CPU. `BeginFrame` waits on it the next time this frame comes round.

When present returns `OUT_OF_DATE` or `SUBOPTIMAL`, or the window reports a resize, the swapchain is marked outdated. The next `BeginFrame` waits for the GPU to be idle and recreates the swapchain, tells ImGui the new image count, and resizes the raytracer's images (which restarts accumulation).

> **Try it: check the synchronization**
>
> Build Debug and run `VK_LAYER_VALIDATE_SYNC=1 ./build-debug/bin/FlyTracer`. The validation layer then tracks every access to every image and buffer and reports hazards such as a write after read without a barrier. A correct frame prints nothing. As an exercise, work out which hazard each barrier in Table 2 prevents, then remove one and see what the layer says.

12 ImGui with dynamic rendering

Dear ImGui draws triangles with a graphics pipeline, which needs a colour attachment. Classic Vulkan required a `VkRenderPass` and a `VkFramebuffer` per swapchain image for that, both recreated on every resize. Since Vulkan 1.3, *dynamic rendering* replaces them with one call that names the attachment directly:

```
const VkRenderingAttachmentInfo colorAttachment{
    .sType = VK_STRUCTURE_TYPE_RENDERING_ATTACHMENT_INFO,
    .imageView = view,                                          // the swapchain image's view
    .imageLayout = VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL,
    .loadOp = VK_ATTACHMENT_LOAD_OP_LOAD,                       // keep the raytraced image
    .storeOp = VK_ATTACHMENT_STORE_OP_STORE,
};
const VkRenderingInfo renderingInfo{
    .sType = VK_STRUCTURE_TYPE_RENDERING_INFO,
    .renderArea = {{0, 0}, extent},
    .layerCount = 1,
    .colorAttachmentCount = 1,
    .pColorAttachments = &colorAttachment,
};
vkCmdBeginRendering(cmdBuffer, &renderingInfo);
ImGui_ImplVulkan_RenderDrawData(ImGui::GetDrawData(), cmdBuffer);
vkCmdEndRendering(cmdBuffer);
```

`LOAD_OP_LOAD` is essential: `CLEAR` or `DONT_CARE` would throw away the raytraced pixels under the

UI. At initialization ImGui is told the swapchain's colour format (`UseDynamicRendering` with a `VkPipelineRenderingCreateInfo`), so it can build its pipeline without a render pass. Because ImGui keeps a pointer to that format, `ImGuiLayer` stores it in a member that lives as long as ImGui does.

13 Measuring: GpuTimer and Benchmark

`GpuTimer` writes a timestamp before and after the dispatch into a query pool (two queries per frame in flight). The results are read back when the frame's slot is reused, after its fence: reading earlier would stall the CPU. Some drivers resolve all timestamps of a command buffer to the same value (KosmicKrisp on macOS does); the timer then reports nothing and the GUI says "not measurable with this driver".

The benchmark (`--benchmark 200`) needs no window. It creates a headless `VulkanContext` and a `RaytracingPass`, and for each scene advances the scene with a fixed time step, records the dispatch with a `CommandSubmitter` and measures the time from submit to completion. An empty submission is measured first, so you can tell how much of a frame is overhead. Compare at 1080p or more: small images are dominated by that overhead.

PART III

The GPU Side

14 The compute shader

14.1 Files

File	Contents
raytracer.comp	`main`: the camera ray, the bounce loop, accumulation and output
flyfish.glsl	FlyFish's types and products in GLSL, same coefficients and signs: join, meet, inner, geometric product, sandwich
common.glsl, bindings.glsl	GPU structs (matching GpuScene.h), descriptor bindings and push constants
ray.glsl	The ray: a point and a direction, and where a ray meets a plane
intersections.glsl	Ray tests for spheres, planes and boxes; the watertight Plücker test for triangle edges
traversal.glsl	The two-level BVH traversal
brdf.glsl, shading.glsl	Sky, BRDFs (Lambert, Blinn-Phong, Cook-Torrance), lights, shadow rays, reflections
random.glsl, output.glsl, debug_lines.glsl	Random numbers, ACES tone mapping and sRGB encoding, the debug line overlay

14.2 One invocation per pixel

```
layout(local_size_x = WORKGROUP_SIZE, local_size_y = WORKGROUP_SIZE) in;

void main() {
    ivec2 pixel = ivec2(gl_GlobalInvocationID.xy);
    ivec2 size = imageSize(outputImage);
    if (pixel.x >= size.x || pixel.y >= size.y) return;

    // The first sample goes through the pixel center, later ones are spread over the pixel
    uint rng = seedRandom(pixel, pc.frameIndex);
    vec2 jitter = pc.sampleIndex == 0u ? vec2(0.5) : vec2(random(rng), random(rng));
    // ...
    vec3 radiance = traceRadiance(cameraRay(vec2(pixel) + jitter, vec2(size)), pixelAngle, rng,
     hitDistance);

    // Running average of all samples since the view or scene last changed
    vec3 average = radiance;
    if (pc.sampleIndex > 0u) {
        vec3 previous = imageLoad(accumulationImage, pixel).rgb;
        average = previous + (radiance - previous) / float(pc.sampleIndex + 1u);
    }
    imageStore(accumulationImage, pixel, vec4(average, 1.0));

    vec3 color = toneMapACES(average * pc.exposure);
    color = drawDebugLines(cameraRay(vec2(pixel) + 0.5, vec2(size)), hitDistance, pixelAngle, color);
    imageStore(outputImage, pixel, vec4(linearToSrgb(color), 1.0));
}
```

The running average is the reason a still image becomes smooth after a moment: every frame adds one jittered sample per pixel, which anti-aliases edges and averages the noise of glossy reflections. This is also why `RaytracingPass` must notice when anything changes and reset `sampleIndex`: otherwise a moving object would leave a trail.

14.3 The camera is a motor

```
Ray cameraRay(vec2 pixel, vec2 imageSize) {
    vec2 ndc = 2.0 * pixel / imageSize - 1.0;
    float aspect = imageSize.x / imageSize.y;
    Motor camera = Motor(pc.cameraRotor.x, pc.cameraRotor.yzw, pc.cameraDual.xyz, pc.cameraDual.w);
    TriVector towards = direction(vec3(ndc.x * aspect * pc.tanHalfFov, -ndc.y * pc.tanHalfFov, -1.0));
    return Ray(sandwich(camera, point(vec3(0.0))), normalized(sandwich(camera, towards)));
}
```

In the camera's own frame every ray starts at the origin and looks down $-z$. The camera motor, built on the CPU from joins of eye, target and up, moves the origin and the direction into the world with the same sandwich $M\,X\,\widetilde{M}$ the C++ side uses. No view matrix exists anywhere.

14.4 Rays into instances

To test a ray against an instance, the shader moves the *ray* into the mesh's object space instead of moving the mesh into the world: undo the mirror (a reflection is its own inverse), sandwich with the reversed motor, divide by the scale. The direction is not renormalized, so a distance t along the ray means the same in both spaces and hits from different instances can be compared directly.

```
Ray toObjectSpace(Ray ray, MeshInstance instance) {
    if (instance.hasMirror != 0u) {
        Vector mirror = Vector(instance.mirror.xyz, instance.mirror.w);
        ray = Ray(reflectInPlane(mirror, ray.origin), reflectInPlane(mirror, ray.direction));
    }
    Motor toObject = reverse(instanceMotor(instance));
    TriVector origin = sandwich(toObject, ray.origin);
    TriVector towards = sandwich(toObject, ray.direction);
    return Ray(point(origin.xyz / instance.scale), direction(towards.xyz / instance.scale));
}
```

Traversal keeps a small stack of node pairs with the distance at which the ray enters them. Pairs entered beyond the closest hit so far are skipped, and the nearer child is visited first, so most of the tree is never touched. Shadow rays use the same traversal in *any-hit* mode, which stops at the first hit.

15 Testing and debugging

Executable	Tests	GPU
FlyTracer_Test	BVH, PGA helpers, camera, input, GPU encoding, meshes, scene encoding	no
FlyTracer_ConfigTest	The config file parser	no
FlyTracer_ShaderTest	flyfish.glsl against FlyFish; GLSL struct layouts, bindings and push constants against GpuScene.h	yes
FlyTracer_SceneTest	A golden image of every registered scene	yes
FlyTracer_RenderTest	A golden image of a test scene, and the benchmark	yes

Golden image tests render a scene headless and compare it with a reference image in tests/reference. When a reference is missing it is recorded, so after an intended change in the look you delete the reference and run the test again.

For Vulkan problems, three tools in increasing order of effort:

1. A **Debug build** enables the validation layer: wrong parameters, missing features and leaked objects are reported immediately, with the name of the rule that was broken.
2. `VK_LAYER_VALIDATE_SYNC=1` adds synchronization validation (missing or wrong barriers).
3. `FLYTRACER_GPU_VALIDATION=1` adds GPU-assisted validation, which instruments the shader to catch out-of-bounds buffer and descriptor accesses.

> **Pitfall: stale shaders**
> Shaders are compiled by the build. When you edit a GLSL file, rebuild before you run, or you test the old SPIR-V. A shader that fails to compile fails the build, with `glslc`'s message pointing at the line.

16 Exercises

1. **A spinning scene.** Write a scene with a mesh that turns about a line that does not pass through the origin. Build the line as the join of two points and the motor with `Motor::Rotation`. Draw the line with `DrawDebugLine`.
2. **Follow a frame.** Set breakpoints in `BeginFrame`, `RaytracingPass::Record` and `EndFrame`. In which order are the fence wait, the acquire, the encoding and the submit executed? Where would the CPU block if the GPU were slow?
3. **Workgroup size.** Change `WORKGROUP_SIZE` in ShaderInterface.h to 4 and to 16 and run the benchmark at 1080p. Why does one header change both the C++ dispatch and the shader?
4. **Present modes.** Run with and without `--no-vsync` and compare the printed swapchain line and the frame rate. What does `MAILBOX` do differently from `FIFO`?
5. **Three frames in flight.** What has to change for `kFramesInFlight = 3`? Check the descriptor pool sizes, `FrameBuffers`, `GpuTimer` and the benchmark's `firstMeasured`.
6. **A new per-frame buffer.** Add a binding to ShaderInterface.h, a struct with `static_asserts` to GpuScene.h and its twin to common.glsl, a buffer to bindings.glsl, a vector to `FrameData`, and one `writeFrameBuffer` call. Then run `FlyTracer_ShaderTest`: does it catch a mismatch you introduce on purpose?

A Vulkan calls by class

Every Vulkan function the engine calls, grouped by where it is called. The `vkDestroy*` functions are called by the `DeviceHandle` aliases.

Where	Calls
VulkanContext	`vkCreateInstance`, `vkEnumerateInstanceLayerProperties`, `vkCreateDebugUtilsMessengerEXT`, `vkEnumeratePhysicalDevices`, `vkGetPhysicalDeviceProperties`, `vkGetPhysicalDeviceQueueFamilyProperties`, `vkGetPhysicalDeviceSurfaceSupportKHR`, `vkCreateDevice`, `vkGetDeviceQueue`, `vkCreateCommandPool`, `vkDeviceWaitIdle`
Swapchain	`vkGetPhysicalDeviceSurfaceCapabilitiesKHR`, `vkGetPhysicalDeviceSurfaceFormatsKHR`, `vkGetPhysicalDeviceSurfacePresentModesKHR`, `vkCreateSwapchainKHR`, `vkGetSwapchainImagesKHR`
VulkanHelpers	`vkCreateBuffer`, `vkCreateImage`, `vkCreateImageView`, `vkGetBufferMemoryRequirements`, `vkGetImageMemoryRequirements`, `vkGetPhysicalDeviceMemoryProperties`, `vkAllocateMemory`, `vkBindBufferMemory`, `vkBindImageMemory`, `vkMapMemory`, `vkCreateSemaphore`, `vkCreateFence`, `vkCreateDescriptorSetLayout`, `vkCreateDescriptorPool`, `vkAllocateDescriptorSets`, `vkUpdateDescriptorSets`, `vkCreatePipelineLayout`, `vkCreateShaderModule`, `vkCreateComputePipelines`, `vkAllocateCommandBuffers`, `vkResetCommandBuffer`, `vkBeginCommandBuffer`, `vkEndCommandBuffer`, `vkQueueSubmit2`, `vkQueueWaitIdle`, `vkFreeCommandBuffers`, `vkCmdPipelineBarrier2`, `vkCmdCopyBuffer`, `vkCmdCopyBufferToImage`, `vkCmdBlitImage`
RaytracingPass	`vkCreateSampler`, `vkCmdBindPipeline`, `vkCmdBindDescriptorSets`, `vkCmdPushConstants`, `vkCmdDispatch`
VulkanRenderer	`vkWaitForFences`, `vkResetFences`, `vkAcquireNextImageKHR`, `vkCmdBlitImage`, `vkQueuePresentKHR`
ImGuiLayer	`vkCmdBeginRendering`, `vkCmdEndRendering` (the rest is inside `imgui_impl_vulkan`)
GpuTimer	`vkCreateQueryPool`, `vkCmdResetQueryPool`, `vkCmdWriteTimestamp2`, `vkGetQueryPoolResults`

B Further reading

- docs/pga-tracer.md: the mathematics of the tracer (camera, rays, triangles, instances, shading) in more depth.
- docs/scenes/: the student guide to writing scenes.
- The Vulkan specification, chapter “Synchronization and Cache Control”, for the exact meaning of stages and access masks.
- S. De Keninck et al., “Clean up your Mesh!” (2025), for the mesh measures in `Mesh`.